

eJual Development Standards

Single-file handoff guide for developers and AI-assisted IDEs continuing eJual after Product and Catalogue Management.

Project	eJual clothing e-commerce system using Payload CMS
Current status	Product and Catalogue Management is complete
Next work	Cart and Checkout, then Order and Payment
Primary UI system	Mantine UI with Geist font and eJual theme
Backend source of truth	Payload CMS collections with PostgreSQL and Supabase S3 media

Use this PDF as the starting context when asking Codex, Claude, Gemini, or another AI coding assistant to continue eJual. The assistant should follow these practices before making changes.

1. AI Operating Instructions

When an AI assistant continues this project, these rules are mandatory:

- Read and follow **AGENTS.md** before changing code.
- Use Ponytail Mode: choose the simplest working solution, reuse existing helpers and patterns, avoid speculative abstractions, and prefer deletion over addition when safe.
- Use the Payload CMS skill or equivalent Payload documentation whenever editing collections, access control, Payload config, hooks, Local API calls, auth, or storage.
- Use the UI Professional rules for every UI/UX task: route design, layout, responsiveness, component choice, visual verification, broken UI behavior, or consistency review.
- Ask clarification when requirements affect data flow, ownership, checkout/payment behavior, security, or UI meaning. Do not guess on high-impact behavior.
- Before finalizing code changes, run **pnpm exec tsc --noEmit**. Run **pnpm run build** for route, schema, or UI changes. For UI changes, verify desktop and mobile in a browser.

Required AI posture: act like a pragmatic senior engineer. Make surgical changes, do not refactor unrelated files, do not revert user changes, and report any test or verification step that could not be completed.

2. Architecture And Stack

Area	Standard
Framework	Next.js App Router with Payload CMS 3
Database	PostgreSQL through @payloadcms/db-postgres
UI	Mantine UI 9 is the default component system
Typography	Geist Sans and Geist Mono
Auth	Payload auth on users collection, stored in payload-token cookie
Media	Payload media collection, local fallback, Supabase S3-compatible storage when env is configured
Admin	/admin is Payload admin only and requires role === admin

Buyer and seller app routes must not show Payload website-template chrome. Current app workspaces hide Payload header, footer, and admin bar through body workspace classes and workspace root classes.

Legacy template routes such as /, /search, and /posts redirect to /shop so the buyer-facing experience remains consistent with eJual.

3. UI/UX Standards

Mantine components are mandatory by default. Build custom components only as small wrappers around Mantine when repeated behavior requires it.

- Use AppShell for buyer/seller shells, Container for page width, Stack and Group for layout, Card/Table/Tabs/Modal for surfaces, and Mantine form inputs for data entry.
- Keep radius, spacing, typography, and color aligned with the current Mantine provider theme: teal primary color, md radius, Geist font family.
- Every UI must be responsive. Check mobile and desktop for overflow, overlapping text, awkward wrapping, and misaligned controls.
- Every app page needs clear title, short orientation copy, visible signed-in user state when applicable, and one dominant primary action.
- Use labels and icons together for status. Color alone is not enough.
- Seller pages should feel like quiet dashboards. Buyer shop should feel like a clothing product grid.
- Product cards must keep image, description, variant count, stock, and price aligned regardless of text length.
- Do not auto-assign category icons unless categories explicitly store icon metadata.
- Empty states must state what is missing and provide one clear next action.

Current key shells: **ShopShell** owns buyer browsing layout and buyer/seller mode actions. **SellerShell** owns seller navigation, logout, and the Shop as Buyer link.

4. Current eJual Collections And Relationships

Collection	Purpose	Key fields / relationships
users	Payload auth users	fullName, email, phoneNumber, profileImage -> media, role, accountStatus
categories	Clothing category taxonomy	categoryName, title, description, slug
media	Uploaded assets	alt, caption, upload file; Supabase S3 when configured
products	Seller-owned listings	seller -> users, category -> categories, productName, description, basePrice, status, featured
product-variants	Purchasable variations	product -> products, color, size, additionalPrice
product-images	Product-level image ordering	product -> products, image -> media, displayOrder
inventory	Variant stock record	variant -> product-variants unique, stockQuantity, reservedQuantity
carts	One cart per user	user -> users unique
cart-items	Rows in cart	cart -> carts, variant -> product-variants, quantity
orders	Checkout order header	buyer -> users, shippingAddress -> addresses, paymentMethod, paymentStatus, orderStatus, subtotal, shippingFee, totalAmount, orderDate
order-items	Order line items	order -> orders, variant -> product-variants, seller -> users, quantity, unitPrice, subtotal
addresses	Buyer shipping addresses	user -> users, receiverName, phoneNumber, address lines, city, state, postcode, country, isDefault
notifications	User messages	user -> users, title, message, type, isRead

5. Backend And Data Practices

- Server reads should use Payload Local API from Server Components.
- Mutations should use server actions or route handlers.
- User-scoped Payload operations must pass user and overrideAccess: false.
- Seller mutations must verify product ownership before editing related product, variant, image, or inventory data.
- Buyers must not see or buy their own products in /shop or future cart flows.
- reservedQuantity is system-managed. Sellers must not edit it manually and product creation must initialize it to 0.
- Product creation currently creates product, one or more variants, inventory rows, and product images together.
- After mutations, revalidate affected routes with revalidatePath.
- Use structured Payload queries rather than ad hoc filtering where access or ownership matters.

Important security rule: Payload Local API bypasses access control by default. For operations performed on behalf of a user, pass overrideAccess: false with the authenticated user.

6. Authentication And Access Rules

- /login and /register are the app-facing auth routes and redirect successful users to /shop.
- /admin/login is only for Payload admin users.
- /admin access requires role === admin. Customers must be redirected to the forbidden page.
- Non-admin users should receive a clear denial page with a path back to shop or seller workspace.
- Suspended users may view seller pages but seller mutations must be blocked.
- Use getMeUser for required authentication and getOptionalMeUser for optional user context.
- Do not expose another seller's product edit page by ID. Query by both product ID and current seller ID.

7. Component Interaction Standards

Surface	Current behavior
/shop	Buyer mode. Lists available products from all sellers except the signed-in user's own products.
/seller/products	Seller mode. Lists only products owned by the signed-in user.
/seller/products/new	Creates product details, variants, initial stock, and product-level images.
/seller/products/[id]/edit	Edits owned product details, variants, product images, and stock only.
/admin	Payload admin. Not part of buyer/seller app UI.

Forms must display clear failure states. Destructive or irreversible future actions should use confirmation. Loading/saving states should be visible for async work.

Product image ordering is seller-controlled at product level through product-images.displayOrder. Images are not variant-specific in the current model.

8. Guidance For Cart, Checkout, Order, And Payment

- Cart/Checkout must use existing carts, cart-items, inventory, addresses, products, and product-variants.
- Do not add a new cart collection unless existing collections cannot represent the required behavior.
- Cart must prevent adding the current user's own product.
- Cart logic should respect stockQuantity and reservedQuantity. reservedQuantity is the system reservation value for items held in carts or checkout flow.
- Checkout subtotal should use `product.basePrice + productVariant.additionalPrice`, multiplied by quantity.
- Checkout must collect or select an address from addresses before creating an order.
- Order/Payment must create orders and order-items, preserve seller attribution on each order item, and update `paymentStatus` and `orderStatus` explicitly.
- Order line item prices should be copied into `unitPrice/subtotal` at checkout time so future product price edits do not rewrite historical orders.
- Use notifications for meaningful system, order, or promotion messages after the related workflows exist.

9. Verification Checklist

Check	Required result
TypeScript	<code>pnpm exec tsc --noEmit</code> passes
Build	<code>pnpm run build</code> passes after route/schema/UI changes
Browser routes	<code>/shop</code> , <code>/login</code> , <code>/register</code> , <code>/seller/products</code> , <code>/seller/products/new</code> , edit pages, <code>/admin</code> , forbidden, not-found
Responsive UI	Desktop and mobile show no horizontal overflow, clipped text, layout overlap, or confusing wrapping
Console/network	No unexpected console errors or non-2xx network failures
Payload chrome	Buyer/seller/auth pages do not show Payload header, footer, or admin bar
Access	Non-admin users cannot access <code>/admin</code> ; sellers cannot edit products they do not own

10. AI Context Prompt To Use With This PDF

When uploading this PDF to an AI-assisted IDE, include a short instruction like this:

You are continuing the eJual Payload CMS clothing e-commerce project. Follow this PDF and AGENTS.md exactly. Use Mantine UI first, preserve eJual UI consistency, enforce ownership and admin access rules, use Payload Local API correctly with `overrideAccess: false` for user-scoped actions, and verify with TypeScript, build, and browser checks. Ask before changing schemas or security-sensitive behavior.

The PDF should be treated as development standards, not as a feature request by itself. Future feature prompts decide what to build; this document decides how to build it.

11. Do Not Break These Invariants

- Do not make Payload website-template pages the buyer storefront again.
- Do not bypass ownership checks for seller product edit or mutation paths.
- Do not let customers access /admin.
- Do not let a seller manually edit reservedQuantity.
- Do not create custom UI components where Mantine already provides a suitable component.
- Do not introduce new collections for cart/order work unless the existing model is proven insufficient.
- Do not rely on color-only status communication.
- Do not finish UI work without mobile and desktop verification.